

# pa7uiclf

**Ciência da Computação**

Teoria dos Grafos

07/11/2025

# Teoria dos Grafos

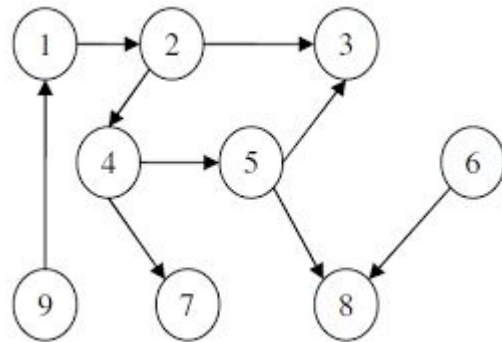
- TAD Grafo
- Tipos de Grafos
- Importância de Grafos
- Métodos de travessia do Grafo
- Matrizes
- Algoritmos de busca
- Exercícios

O **TAD Grafo (Tipo Abstrato de Dado Grafo)** é uma **estrutura abstrata** usada para representar **relações entre elementos (vértices)** por meio de **ligações (arestas)**.

Ele define **operações e propriedades** que permitem manipular e consultar grafos, sem se preocupar com detalhes de implementação.

## Componentes principais

- **Vértices (ou nós):** elementos do conjunto que representam objetos ou entidades.
- **Arestas:** conexões entre os vértices, podendo ser:
  - **Orientadas (arcos):** têm direção (ex:  $A \rightarrow B$ ).
  - **Não orientadas:** não têm direção (ex:  $A - B$ ).
- **Pesos (opcional):** valores associados às arestas (ex: custo, distância, tempo).



## Grafo $G=(V,E)$

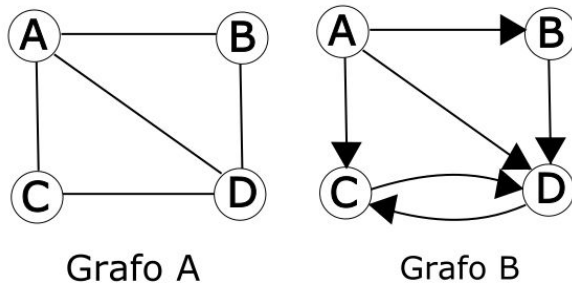
- V: conjunto de **vértices** (ou nós)
- E: conjunto de **arestas** (ligações entre vértices)

# Tipos de Grafos

- Não direcionado / Direcionado
- Ponderado / Não ponderado
- Conexo / Desconexo
- Cíclico / Acíclico

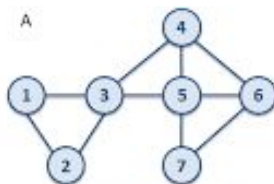
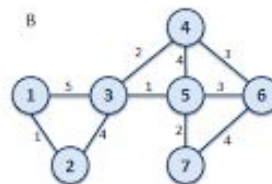
# Tipos de Grafos: Não direcionado / Direcionado

O grafo é *direcionado* quando os relacionamentos têm uma direção específica que importa. Por exemplo, imagine uma relação de “seguir” nas redes sociais. Se você segue alguém no Instagram, a direção é clara: você vai de você para a pessoa que segue. Não é recíproco até que a pessoa te siga de volta. Ou seja, é uma via de mão única, onde a direção importa. Por outro lado, o grafo é *não direcionado* quando os relacionamentos são mútuos ou bidirecionais. Por exemplo, imagine uma solicitação de amizade no Facebook. Se você é amigo de alguém, essa amizade é mútua, ou seja, ambas as pessoas estão igualmente conectadas. Não importa quem adicionou quem primeiro a relação é bidirecional. Ou seja, a direção não importa.



# Tipos de Grafos: Ponderado / Não ponderado

Um grafo é *ponderado* quando seus relacionamentos possuem valores ou pesos, como, por exemplo, a distância entre duas cidades, o custo de uma viagem ou a força de uma amizade. Já quando todos os relacionamentos de um grafo são considerados iguais, ele é *não ponderado*. Esse comportamento afeta diretamente os algoritmos de busca de caminhos, onde “o caminho mais curto” pode significar aquele com menor custo (como o mais barato ou o mais rápido), em vez de simplesmente o caminho com o menor número de etapas.


$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{bmatrix} \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix}$$

$$A = \begin{bmatrix} 0 & 1 & 5 & 0 & 0 & 0 & 0 \\ 1 & 0 & 4 & 0 & 0 & 0 & 0 \\ 5 & 4 & 0 & 2 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 & 4 & 3 & 0 \\ 0 & 0 & 1 & 4 & 0 & 3 & 2 \\ 0 & 0 & 0 & 3 & 3 & 0 & 4 \\ 0 & 0 & 0 & 0 & 2 & 4 & 0 \end{bmatrix} \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{matrix}$$

# Tipos de Grafos: Denso / Esparso

O grafo é considerado *denso* se possui muitos relacionamentos em relação aos nós. E *esparso* se tem poucos relacionamentos. A maioria dos grafos do mundo real são esparsos, e a densidade pode impactar a escolha e o desempenho dos algoritmos, especialmente no processo de detecção de comunidades.

**Esparso**



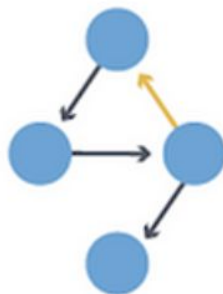
**Denso**



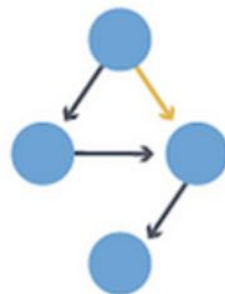
# Tipos de Grafos: Cíclico / Acíclico

O grafo pode conter ciclos, caminhos que começam e terminam no mesmo nó (*cíclico*). Por exemplo, dependências circulares. Ou é *acíclico*, ou seja, não possui ciclos, por exemplo em estruturas de hierarquias organizacionais ou dependências de tarefas de um projeto). Ciclos em um grafo exigem um tratamento especial em alguns algoritmos.

**Cíclico**



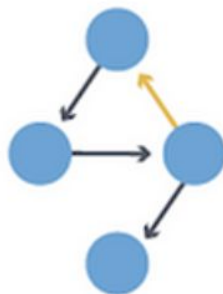
**Acíclico**



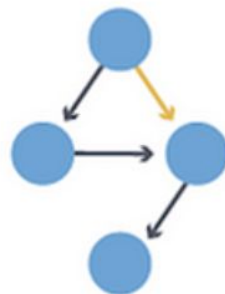
# Tipos de Grafos: Cíclico / Acíclico

O grafo pode conter ciclos, caminhos que começam e terminam no mesmo nó (*cíclico*). Por exemplo, dependências circulares. Ou é *acíclico*, ou seja, não possui ciclos, por exemplo em estruturas de hierarquias organizacionais ou dependências de tarefas de um projeto). Ciclos em um grafo exigem um tratamento especial em alguns algoritmos.

**Cíclico**



**Acíclico**



# Importância de Grafos

Os grafos de dados possibilitam análises detalhadas de relacionamentos, sejam eles diretos ou estruturais, oferecendo insights profundos sobre padrões e conexões. Suas aplicações incluem:

- Mapeamento de relacionamentos em redes sociais.
- Detecção de ameaças cibernéticas.
- Identificação de fraudes financeiras e transações suspeitas.
- Recomendações de produtos e serviços.
- Otimização de rotas logísticas.

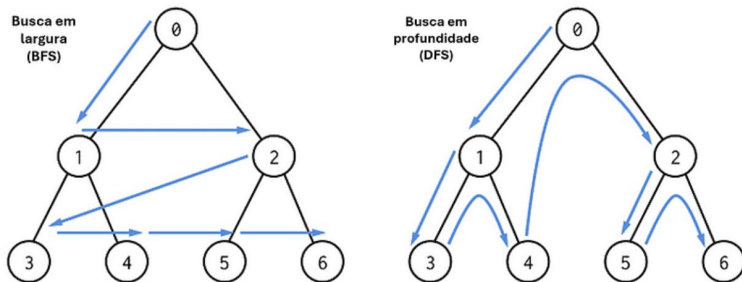
Essas estruturas também são cruciais para abordagens avançadas, como *deep learning*, onde são utilizadas para identificar padrões complexos e treinar modelos de aprendizado de máquina com maior eficiência.

# Métodos de travessia do Grafo

Outro conceito importante para entendermos os algoritmos de grafos é relacionado a travessia do grafo ou graph traversal.

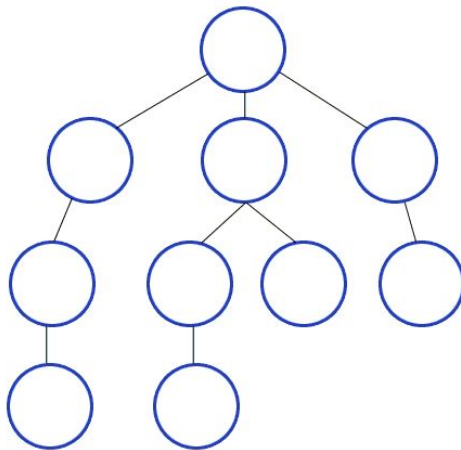
Esse é o processo de visitar os nós de um grafo de forma ordenada seguindo seus relacionamentos. Imagine que acabamos de abrir o Waze e colocamos como destino a Avenida Paulista. O Waze, então, irá traçar a rota, mostrando as ruas e avenidas disponíveis para chegar ao destino. Esse processo de navegar entre ruas e avenidas é semelhante à travessia de grafos, onde você explora as conexões (ruas) entre os pontos de partida e chegada (nós), seguindo um caminho até o destino.

Os dois algoritmos de travessia de grafos mais básicos são a busca em largura (BFS) e a busca em profundidade (DFS).



# Métodos de travessia do Grafo

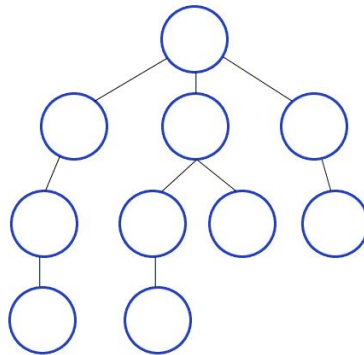
**Busca em Largura (BFS)** : Esse algoritmo percorre o grafo camada por camada. Partindo de um nó, ele visita todos os seus vizinhos imediatos, depois os vizinhos dos vizinhos, e assim por diante. É como quando atiramos uma pedra em um lago e as ondas se expandem gradualmente em todas as direções. A cada “onda”, o algoritmo visita uma nova camada de nós, até explorar todo o grafo



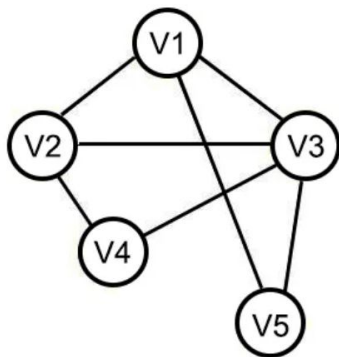
# Métodos de travessia do Grafo

**Busca em profundidade (DFS):** Esse algoritmo explora o máximo possível ao longo de um único caminho antes de voltar. Partindo de um nó, ele segue por um vizinho, depois por outro, e assim por diante, até encontrar um beco sem saída, quando retrocede para explorar outros caminhos.

Embora sejam simples, tanto a BFS quanto a DFS são partes fundamentais que servem de base para muitos algoritmos de grafos mais complexos que veremos mais adiante.



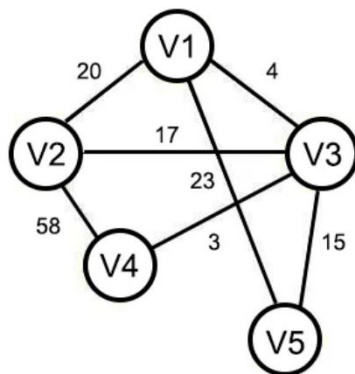
## Matriz de Adjacência



|    | V1 | V2 | V3 | V4 | V5 |
|----|----|----|----|----|----|
| V1 | 0  | 1  | 1  | 0  | 1  |
| V2 | 1  | 0  | 1  | 1  | 0  |
| V3 | 1  | 1  | 0  | 1  | 1  |
| V4 | 0  | 1  | 1  | 0  | 0  |
| V5 | 1  | 0  | 1  | 0  | 0  |

# Grafo por Matriz

## Matriz de Adjacência grafo ponderado



\*zero é um valor escolhido  
em código para considerar não ter  
nenhuma ligação entre os dois grafos,  
porém se seu grafos tiver zero como um  
valor valido deve se escolher outro valor.

|    | V1 | V2 | V3 | V4 | V5 |
|----|----|----|----|----|----|
| V1 | 0* | 20 | 4  | 0* | 23 |
| V2 | 20 | 0* | 17 | 58 | 0* |
| V3 | 4  | 17 | 3  | 3  | 15 |
| V4 | 0* | 58 | 3  | 0* | 0* |
| V5 | 23 | 0* | 15 | 0* | 0* |

# Algoritmos de busca

**Principais perguntas respondidas por esses tipos de algoritmos:**

- Como essas duas entidades estão conectadas?
- Qual é a rota mais curta/barata/rápida entre A e B?
- Existem ciclos (loops) que incluem determinados nós?
- Qual é o caminho crítico em um processo ou grafo de dependência?
- Qual é o grau de separação entre dois nós?

# Algoritmos de busca

- Algoritmo de Dijkstra (**Dijkstra's Algorithm**): Calcula o caminho mais curto com base nos pesos dos relacionamentos (por exemplo, custo, distância) em grafos com pesos não negativos.
- Algoritmo de busca A\* (A\* Search): É um algoritmo de busca informada para grafos ponderados, usado em roteirização (como em mapas). Ele combina o custo já percorrido com uma estimativa da distância até o destino, encontrando o caminho mais curto de forma mais rápida e eficiente.
- Caminho Mais Curto entre Todos os Pares (**Floyd-Warshall**): Calcula os caminhos mais curtos entre todos os pares de nós no grafo.
- Caminho Mais Curto a partir de uma Única Fonte (**Bellman-Ford**): Encontra os caminhos mais curtos a partir de um nó inicial específico para todos os outros nós alcançáveis.
- O **Caixeiro Viajante** precisa visitar um conjunto de cidades **exatamente uma vez cada** e **retornar à cidade de origem**, percorrendo a **menor distância total possível**

# Exercícios

- [https://graphonline.top/pt/create\\_graph\\_by\\_matrix](https://graphonline.top/pt/create_graph_by_matrix)
- <https://www.metrocptm.com.br/veja-o-mapa-de-estacoes-do-metro-e-cptm/>
- <https://github.com/valdomiro-rompi/grafos>
-